



ローカルLLM環境構築レポート

アプリ開発における実用的なローカルLLMの選定及び環境構築



実施内容サマリー

セッション目標の達成

ミニマムな組み合わせから学習を経て、エージェントワークフロー特化IDEとローカルLLMの統合を目指す

- CLI+llama+ローカルLLM
- ~CLI+LM Studio+ローカルLLM
- ~AntiGravity+LM Studio+ローカルLLM

成果確認

- ✓ GPU 最適化分析
- ✓ llamaサーバー セットアップ
- ✓ LM Studioサーバー セットアップ
- ✓ Antigravity IDE 統合試行
- ✓ Continue 拡張機能統合
- ✓ 環境検証
- ✓ ローカルLLM検証



システム構成

ハードウェア

コンポーネント	仕様
GPU	AMD RX-6900XT (16GB VRAM)
OS	Windows 11 Pro
実装RAM	32GB (DDR4)

ソフトウェアスタック

レイヤー	コンポーネント
CLI	141plot CLI (独自構築 — 開発記は 【付録3】 参照)
IDE	Antigravity IDE v1.107.0
拡張機能	Continue (LM Studio プロバイダー)
LLM サーバー	LM Studio (Port 1234, OpenAI互換API)
モデル	Qwen-2.5-Coder-14B (Q4_K_M量子化)

GPU メモリプロファイル（Qwenの例）

GPU メモリ配分（RX-6900XT 16GB）

16.00 GB VRAM	
Qwen モデル: 12.62 GB	← 78.9% 使用
OS & 予約: 1.00 GB	
自由領域: 2.38 GB	← バッファ (21.1%)

⚙️ 設定と実行方法例

LM Studio 起動コマンド

```
lms server start
```

```
lms load qwen/qwen2.5-coder-14b --gpu max --context-length 32768 --parallel 4 -y
```

自動起動スクリプト

位置: `C:\Users\%141di\start-qwen-optimized.ps1`

このスクリプトを実行すると、LM Studio サーバーが最適設定で自動起動します。

Continue 設定ファイル

位置: `C:\Users\%141di\%.continue\config.yaml`

```
name: My Config
version: 0.0.1
schema: v1
defaultModel: Qwen 2.5 Coder 14B (Local)
models:
  - name: Qwen 2.5 Coder 14B (Local)
    provider: lmstudio
    model: qwen/qwen2.5-coder-14b
    apiBase: http://localhost:1234/v1
disableIndexing: true
```

API エンドポイント

URL: <http://localhost:1234/v1>

タイプ: OpenAI 互換 API



パフォーマンス指標（Qwen-2.5-Coder-14B）

ロード時間

条件	ロード時間	備考
初期ロード	15.56秒	GPU設定なし
GPU max	12.90秒	17.1% 高速化
最適設定（32K コンテキスト）	30.27秒	フル設定

推論パフォーマンス

タスク	推論時間
短いチャット（<50トークン）	2-5秒
コード生成（100-200トークン）	5-10秒
長いコード生成（500+トークン）	15-30秒
自動補完（10-30トークン）	< 2秒



モデルごとの実用性検証

設計～実装までバランスが良い

現環境において、設計～実装までバランスが良いと感じたのはこのモデル。

- `granite-4.0-h-tiny-Q4_K_M.gguf` : 4.23GB

検索・コーディング・テストに実用性を感じた

- `granite-3.3-2b-instruct-q4_k_m.gguf` : 1.55GB
- `granite-3.3-2b-instruct-Q6_K.gguf` : 2.98GB

コーディングは問題ないが偶にフリーズ

- `DeepSeek-V4-Flash-MTP-Q4K-Q8_0-F32.gguf` : 3.81GB
- `deepseek-r1-0528-qwen3-8b-q4_k_m.gguf` : 5.03GB
- `granite-4.1-8b-GGUF/granite-4.1-8b-Q4_K_M.gguf` : 5.35GB
- `granite-4.1-8b-Q4_K_S.gguf` : 5.09GB

推論は良いが手が進まない

- `gemma-4-12B-it-Q4_K_M.gguf` : 7.56GB
- `gemma-4-12b-qat-Q4_0` : 7.15GB
- `Qwen3.5-9B-Q4_K_S.gguf` : 7.22GB

オフロードが重すぎて実用に耐えない

- `Qwen3-14B-Q4_K_M.gguf` : 9.00GB

🌟 利用可能な機能

Antigravity + ローカルLLM

Continue 拡張機能による統合で以下が利用可能。

- **チャット機能** - IDE 内でのテキスト会話
- **自動補完** - Tab キーでのコード生成
- **編集支援** - /edit コマンドでコード改修提案
- **ドキュメント生成** - /doc コマンドで自動ドキュメント作成
- **テスト生成** - /test コマンドでユニットテスト自動生成

141plot CLI + ローカルLLM

Gemini CLI のカスタマイズ版のため、Gemini CLI の機能が利用可能（開発記は [【付録3】自分だけのCLIを作ってみた](#) を参照）。

- **チャット機能** - ターミナル上での対話型セッション（REPL）
- **コード生成・編集** - ファイルの読み書きツールによるコード作成・改修
- **ファイル参照** - @ 指定でファイル・ディレクトリをコンテキストに追加
- **シェルコマンド実行** - ! プレフィックスや承認付きツールでのコマンド実行
- **Web 検索・取得** - Web 検索グラウンディング / URL コンテンツの取得
- **MCP 対応** - MCP サーバー経由での外部ツール統合
- **メモリ機能** - GEMINI.md によるプロジェクトコンテキストの保持
- **非対話モード** - -p オプションでスクリプトからワンショット実行
- **カスタムスラッシュコマンド** - 独自コマンドの定義・実行
- **チェックポイント** - ファイル変更前の状態保存・復元

ローカル実行のメリット

- ✓ **データプライバシー** - コード、入力内容がすべてローカルで処理
- ✓ **無制限利用** - API 呼び出し制限なし
- ✓ **低遅延** - ネットワーク遅延がない
- ✓ **onピークの遅延がない** - 利用集中時間帯でも応答速度が安定
- ✓ **オフライン対応** - インターネット接続不要



主要な知見と教訓

1. IDE 統合パスの選定

Antigravity IDE のネイティブ機能では LM Studio との直接統合が困難でしたが、Continue 拡張機能を使用することで実現できました。

2. Continue の設定ファイル形式

Continue の設定は YAML 形式（`config.yaml`）であり、JSON ではありません。モデル識別子は完全形式（`qwen/qwen2.5-coder-14b`）で指定する必要があります。

3. GPU メモリの効率的な活用

RX-6900XT の 16GB VRAM に対して、Q4_K_M 量子化された 14B モデルを 78.9% の利用率で運用でき、十分な安全マージン（21.1%）を確保できます。ただし体感の応答速度では小型モデルが優位でした（「モデルごとの実用性検証」参照）。

4. Antigravity + Continue でのローカルLLM利用の限界

Antigravity + Continue でのローカルLLM利用は、推論ばかり肥大し、簡易なコーディングタスクには不向きと感じました。

アプリ開発におけるローカルLLMの実用性

- 検索・コーディング・テストといった単機能タスクであれば、2B クラスの小型モデル（granite-3.3-2b 系）でも十分実用に耐える
- 設計～実装まで通しで任せる場合は、モデル性能と応答速度のバランスが重要（現環境では granite-4.0-h-tiny が最適解）
- 8B 超のモデルはフリーズやオフロード負荷が顕著になり、16GB VRAM 環境では実用性が急激に低下する
- IDE 統合（Antigravity+Continue）は推論が肥大しやすく簡易タスクに不向き。CLI 経由の利用の方が軽快に運用できる

クラウドLLMとの比較（速度・性能）

- 純粋な推論性能（コード品質・複雑なタスクの遂行能力）は、クラウドの最新大型モデルには及ばない
- ネットワーク遅延・onピーク時の混雑がないため、自動補完などの短い応答はローカルの方が体感が安定する
- API 利用料・レート制限が一切なく、試行錯誤を無制限に回せる点はクラウドにない強み
- 長文コンテキストや高負荷タスクではロード時間・推論時間が顕著に伸び、クラウドとの性能差が大きく開く

ローカルLLMを利用すべきシーン

- 機密性の高いコード・データを外部に出せない案件（データプライバシー要件がある開発）
- 自動補完・コミットメッセージ生成など、小さな推論を高頻度で回すタスク
- オフライン環境やネットワークが不安定な環境での開発
- API コストを気にせず大量の試行錯誤（プロンプト検証・テスト生成など）を行いたい場合